

Parallel^xepipedoNN^{ub}

lb Computing Trustworthy Robustness Certifications

Merkouris Papamichail[†]

[†]Computer Science Department, University of Crete
Institute of Computer Science, FORTH

Fall, 2025-2026



Presentation's Outline

- 1 Introduction
 - Neural Networks
 - Adversarial Examples
 - Formal NN Verification
- 2 Methodology
 - High-Dimensional Intervals
 - Interval Certifications
 - Algorithms
- 3 Implementation
 - Installation
 - Examples

Presentation's Outline

- 1 Introduction
 - Neural Networks
 - Adversarial Examples
 - Formal NN Verification
- 2 Methodology
 - High-Dimensional Intervals
 - Interval Certifications
 - Algorithms
- 3 Implementation
 - Installation
 - Examples

Artificial Neural Networks

- ▶ Over the past decade Artificial Neural Networks (NNs) have been the *de facto* machine learning (ML) approach.
 - Using subsymbolic representations, able to process raw data.
 - Sequential layers of linear functions (Dense Layers).
 - Non linear *activation* functions (e.g. ReLU).
 - **Universal Approximators**: Approximate any function, at any error bound.

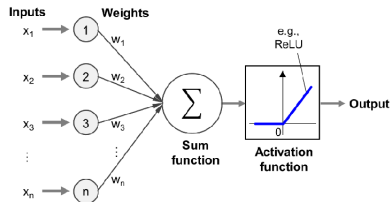


Figure 1: Linear unit $y = \mathbf{w}^T \mathbf{x}$.
Activation ReLU unit: $a(y) = \max(y, 0)$.

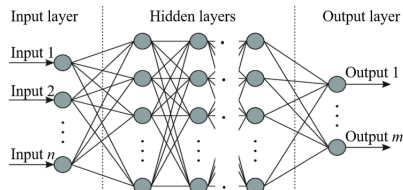


Figure 2: Dense Artificial Neural Network.

Adversarial Examples (1/2)

- ▶ Despite their advantages NN can exhibit highly complex behavior.
 - NNs create complex, *non-convex*, *non-linear* decision surfaces.
 - This complexity produces unexpected irregularities in their predictions.
 - *Adversarial examples*: Slight changes in the input can break the NN's prediction.

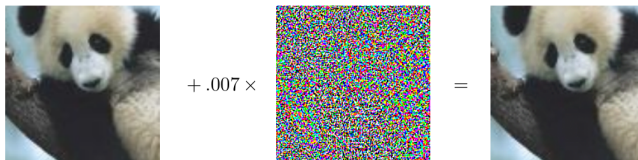


Figure 3: An image of a panda (left) correctly classified. By adding small noise the resulting image (right) gets classified as gibbon. [Goodfellow et al. 2015]

Adversarial Examples (2/2)

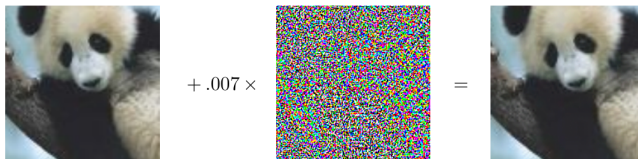


Figure 4: An image of a panda (left) correctly classified. By adding small noise the resulting image (right) gets classified as gibbon. [Goodfellow et al. 2015]

- The two images are *indistinguishable* to the human eye.
- Adversarial examples seem to be *inherited* to neural networks.
- Their existence is *independent* from the network's accuracy.

Intuition

*We say that a NN is **robust** at a given input $\mathbf{x}$, if there are no adversarial examples in the vicinity of $\mathbf{x}$.*

► The two dual problems of adversarial robustness:

- 1 Given a NN, an input $\mathbf{x}$ and an area V , s.t. $\mathbf{x} \in V$. **Certify** that V is *adversarial-free*.
- 2 Given a NN, an input $\mathbf{x}$ and an area V , s.t. $\mathbf{x} \in V$. **Find** an adversarial example $\mathbf{x}'$, s.t. $\mathbf{x}' \in V$.

Intuition

*We say that a NN is **robust** at a given input $\mathbf{x}$, if there are no adversarial examples in the vicinity of $\mathbf{x}$.*

► The two dual problems of adversarial robustness:

- ① *Given a NN, an input $\mathbf{x}$ and an area V , s.t. $\mathbf{x} \in V$. **Certify** that V is adversarial-free.*
- ② **Given a NN, an input $\mathbf{x}$ and an area V , s.t. $\mathbf{x} \in V$. **Find** an adversarial example $\mathbf{x}'$, s.t. $\mathbf{x}' \in V$.**

Testing vs Verification (1/3)

► *Given a NN, an input $\mathbf{x}$ and an area V , s.t. $\mathbf{x} \in V$. **Certify** that V is adversarial-free.*

How do we do that?

- Sample V , test every sample $\mathbf{x}'$ if $\kappa(\mathbf{x}') = \kappa(\mathbf{x})$.
- Formally prove that, for every sample $\mathbf{x}' \in V$ holds $\kappa(\mathbf{x}') = \kappa(\mathbf{x})$.

Testing vs Verification (2/3)

► Given a NN, an input $\mathbf{x}$ and an area V , s.t. $\mathbf{x} \in V$. **Certify** that V is adversarial-free.

How do we do that?

- Sample V , test every sample $\mathbf{x}'$ if $\kappa(\mathbf{x}') = \kappa(\mathbf{x})$.
But, how many samples to take?
 - Assume the standard MNIST dataset, of 28×28 images.
 - These inputs “live” in a 784-dimensional space: $[0, 1]^{784}$.
 - Let's sample this space at a rate of $\delta = 0.1$.
 - This results to 10^{784} samples!!!
 - *There are (waaay) more samples than stars in the galaxy!*

Moreover..

“Program testing can be used to show the presence of bugs, but never to show their absence!” — E. W. Dijkstra

Testing vs Verification (3/3)

- *Given a NN, an input $\mathbf{x}$ and an area V , s.t. $\mathbf{x} \in V$. **Certify** that V is adversarial-free.*

How do we do that?

- Sample V , test every sample $\mathbf{x}'$ if $\kappa(\mathbf{x}') = \kappa(\mathbf{x})$.
- *Formally prove that, for every sample $\mathbf{x}' \in V$ holds $\kappa(\mathbf{x}') = \kappa(\mathbf{x})$.*

NN as a Mixed Integer Linear Program

- ▶ In order to prove a NN's property we represent it as a *Mixed Integer Linear Program (MILP)*.
 - Defining the I/O-relation of a NN as a set of linear inequalities.
 - ReLU activation function involve non-linear, integer constraints.
 - Expanding Linear Programming (LP) formulation to include integer constraints.
 - We solve MILP by fixing the integer values and solving the resulting LP.
 - For an MILP with n binary variables, there are 2^n induced LPs.
 - Solving an MILP is computationally *intractable*.
 - A *NN Verification* oracle solves an MILP representation of a NN *efficiently*.
 - For a set of constraints, it either finds a satisfying assignment (“sat”), or reports “unsat”.

Presentation's Outline

- 1 Introduction
 - Neural Networks
 - Adversarial Examples
 - Formal NN Verification
- 2 Methodology
 - High-Dimensional Intervals
 - Interval Certifications
 - Algorithms
- 3 Implementation
 - Installation
 - Examples

High-Dimensional Intervals (1/2)

Definition: Comparison in $\mathbb{R}^d$

Consider two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^d$. $\mathbf{x} \leq \mathbf{y}$ denotes the *coordinate-wise* comparison of the two vectors, i.e.,

$$\mathbf{x} \leq \mathbf{y} \Leftrightarrow \bigwedge_{i \in [d]} x_i \leq y_i. \quad (1)$$

Definition: Intervals in $\mathbb{R}^d$

Consider two vectors $\mathbf{lb}, \mathbf{ub} \in \mathbb{R}^d$. $[\mathbf{lb}, \mathbf{ub}] \subseteq \mathbb{R}^d$ denotes the set,

$$[\mathbf{lb}, \mathbf{ub}] = \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{lb} \leq \mathbf{x} \leq \mathbf{ub}\}. \quad (2)$$

High-Dimensional Intervals (2/2)

Definition: Intervals in $\mathbb{R}^d$

Consider two vectors $\mathbf{lb}, \mathbf{ub} \in \mathbb{R}^d$. $[\mathbf{lb}, \mathbf{ub}] \subseteq \mathbb{R}^d$ denotes the set,

$$[\mathbf{lb}, \mathbf{ub}] = \{\mathbf{x} \in \mathbb{R}^d \mid \mathbf{lb} \leq \mathbf{x} \leq \mathbf{ub}\}. \quad (3)$$

- Geometrically, intervals correspond to *axis-aligned, hyper-rectangles*.
- Intervals have been chosen due to their nice computational properties:
 - **Intersection** $[\mathbf{a}, \mathbf{b}] \cap [\mathbf{c}, \mathbf{d}]$: $O(d)$
 - **Membership** $[\mathbf{a}, \mathbf{b}] \subseteq [\mathbf{c}, \mathbf{d}]$: $O(d)$
- Extensive use in CS, e.g. Computer Graphics, Vision, etc.

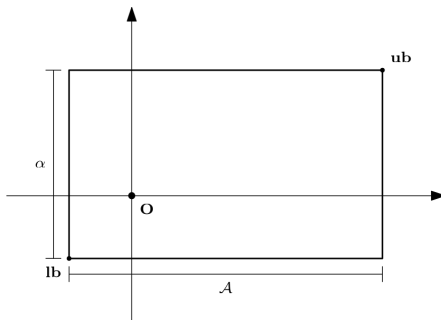


Figure 5: Measures on intervals.

- Minimum Edge Length: $\alpha([\mathbf{lb}, \mathbf{ub}]) = \min_{i \in [d]} ub_i - lb_i$.
- Maximum Edge Length: $\mathcal{A}([\mathbf{lb}, \mathbf{ub}]) = \max_{i \in [d]} ub_i - lb_i$.

Definition of a Sound Interval Certification

Definition

Let $\mathbf{x} \in \mathbb{F}$ be an input, and $\mathcal{C}$ is a finite set of classes. Let $\kappa: \mathbb{F} \rightarrow \mathcal{C}$ be a MLP classifier, s.t. $\mathbb{F} \subseteq \mathbb{R}^d$. We say that the interval $[\mathbf{lb}, \mathbf{ub}] \subseteq \mathbb{F}$ is a *sound* interval certification, if the following holds:

$$\begin{aligned} \mathbf{x} &\in [\mathbf{lb}, \mathbf{ub}] \\ \forall \mathbf{x}' \in [\mathbf{lb}, \mathbf{ub}], \quad \kappa(\mathbf{x}') &= \kappa(\mathbf{x}). \end{aligned} \tag{4}$$

- For each input in $[\mathbf{lb}, \mathbf{ub}]$ the prediction of $\mathbf{x}'$ agrees with the prediction of $\mathbf{x}$, w.r.t. the given MLP $\kappa(\cdot)$.
- A sound certification *always* exists, e.g. the *trivial* $[\mathbf{x}, \mathbf{x}]$.
- We are interested in *maximally* sound certifications, w.r.t. set inclusion $\subseteq$.

Geometric Representation

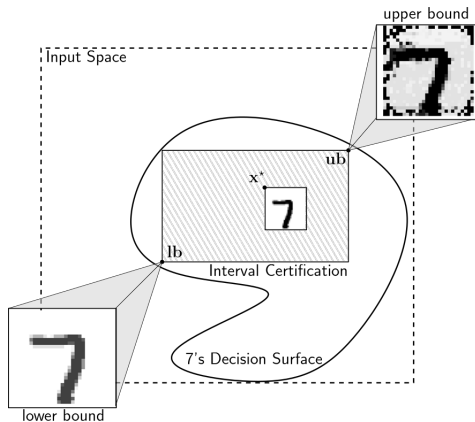


Figure 6: Geometric representation of a interval robustness certification.

Uniform vs General Interval Certifications

Remarks:

- We demand $\mathbf{x} \in [\mathbf{lb}, \mathbf{ub}]$.
- Thus, the interval can be re-written as $[\mathbf{x} - \underline{\mathbf{d}}, \mathbf{x} + \overline{\mathbf{d}}]$, where:
 - $\underline{\mathbf{d}} = \mathbf{x} - \mathbf{lb}$, $\overline{\mathbf{d}} = \mathbf{ub} - \mathbf{x}$.
- We call the above time of intervals general.
- When $\underline{\mathbf{d}} = \overline{\mathbf{d}} = \rho \mathbf{1}$, for some $\rho > 0$, we call the interval *uniform*.
- Uniform intervals correspond to ℓ_∞ -spheres.
- Recall $\ell_\infty(\mathbf{x}) = \|\mathbf{x}\|_\infty = \max_{i \in [d]} |\mathbf{x}_i|$.
- General intervals have $2d$ searchable parameters.
- Uniform intervals have only 1 searchable parameter, the radius ρ .

ParallelepipedoNN Algorithms

	Algorithm	Command	Interval Type	Complexity
1	Cyc. Bottom-Up Dichotomic	-al c-bu-d	Uniform	$O(\log(\mathcal{A}(\mathbb{F})))$
2	Top-Down	-al td	General	$O(d \cdot \mathcal{A}(\mathbb{F})/\delta)$

Remarks:

- $\mathbb{F}$, denotes the input space, $\mathbb{F} = [\underline{\mathbf{U}}, \overline{\mathbf{U}}]$.
- $\mathcal{A}(\mathbb{F})$ denotes the *diameter* of $\mathbb{F}$, w.r.t. the ℓ_∞ -norm.
 - $\mathcal{A}(\mathbb{F}) = \|\overline{\mathbf{U}} - \underline{\mathbf{U}}\|_\infty = \max_{i \in [d]} |\overline{U}_i - \underline{U}_i|$
- $\delta > 0$ denotes a percision constant. Usually, $\delta = 0.1$.

ParallelepipedoNN Algorithms

	Algorithm	Command	Interval Type	Complexity
1	Cyc. Bottom-Up Dichotomic	-al c-bu-d	Uniform	$O(\log(\mathcal{A}(\mathbb{F})))$
2	Top-Down	-al td	General	$O(d \cdot \mathcal{A}(\mathbb{F})/\delta)$

Remarks:

- Algorithm 1 employs a *dichotomic* (binary) search to determine the maximum radius ρ . This makes it significantly faster.
- Using uniform intervals makes Algorithm 1 less precise.

ParallelepipedoNN Algorithms

	Algorithm	Command	Interval Type	Complexity
1	Cyc. Bottom-Up Dichotomic	-a1 c-bu-d	Uniform	$O(\log(\mathcal{A}(\mathbb{F})))$
2	Top-Down	-a1 td	General	$O(d \cdot \mathcal{A}(\mathbb{F})/\delta)$

Remarks:

- Algorithm 2 employs a *top-down* search policy to determine a maximally sound interval certification.
- Algorithm 2 is able to handle general intervals, making much more precise, but also much slower.

Choosing the Correct Algorithm (1/2)

- ▶ Choosing the correct algorithm *does* makes a difference.



Figure 7: Lower (left) and the upper bound (right), *Cyclic Bottom-Up Dichotomic Algorithm*.



Figure 8: Lower (left) and the upper bound (right), *Top-Down Algorithm*.

Choosing the Correct Algorithm (2/2)

- ▶ Choosing the correct algorithm *does* makes a difference.



Figure 9: Lower (left) and the upper bound (right), *Cyclic Bottom-Up Dichotomic Algorithm*.



Figure 10: Lower (left) and the upper bound (right), *Top-Down Algorithm*.

Avg. CPU time:

- *Cyc. Bottom-Up Dich.* $\simeq 21$ (sec)

- *Top-Down* $\simeq 51$ (min)

Avg. Min. Edge Length:

- *Cyc. Bottom-Up Dich.* $\simeq 0.07$

- *Top-Down* $\simeq 0.13$

A Top Down Algorithm: Initialization

- 1 We begin with the *most general* interval.

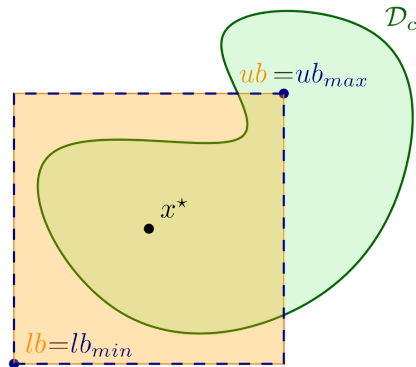


Figure 11

A Top Down Algorithm: Query a NN Verification Oracle

- ① We begin with the *most general* interval.
- ② We query a NN Verification oracle:
 - If the interval is sound, oracle returns “yes”.
 - Else return a counter-example x^c .

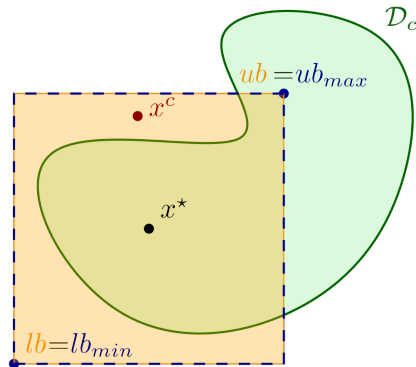


Figure 12

A Top Down Algorithm: Update Interval

- ① We begin with the *most general* interval.
- ② We query a NN Verification oracle:
 - If the interval is sound, oracle returns “yes”.
 - Else return a counter-example x^c .
- ③ We update the interval in order to *exclude* the counter-example.
 - Minimum change to the interval.

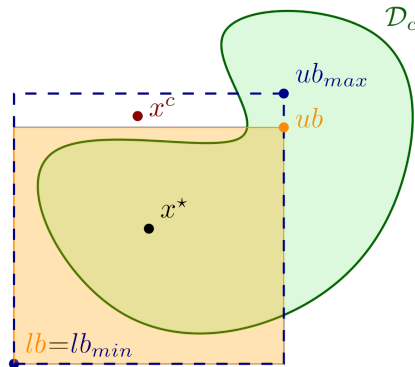


Figure 13

A Top Down Algorithm: Termination Criterion

- 1 We begin with the *most general* interval.
- 2 We query a NN Verification oracle:
 - If the interval is sound, oracle returns “yes”.
 - Else return a counter-example x^c .
- 3 We update the interval in order to *exclude* the counter-example.
 - Minimum change to the interval.
- 4 Repeat until *robustness* is reached.

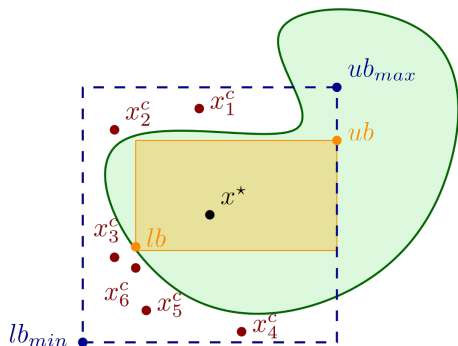


Figure 14

Presentation's Outline

- 1 Introduction
 - Neural Networks
 - Adversarial Examples
 - Formal NN Verification
- 2 Methodology
 - High-Dimensional Intervals
 - Interval Certifications
 - Algorithms
- 3 Implementation
 - Installation
 - Examples

Pipeline

- 1 We begin by training a simple TensorFlow NN model, on MNIST.
- 2 We export the model as Open Neural Network Exchange (ONNX) format.
- 3 We load the ONNX representation in the **Marabou** NN verifier.
- 4 We compute an certification with the desired algorithm.

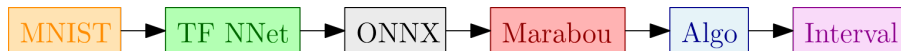


Figure 15: Implementation's Pipeline

Installing Anaconda (Skip if Already Installed)

► On an Ubuntu machine use the following commands:

① Download anaconda:

```
> curl -O https://repo.anaconda.com/archive/  
Anaconda3-2025.12-2-Linux-x86_64.sh
```

② Install:

```
> bash /Anaconda3-2025.12-2-Linux-x86_64.sh
```

The above commands should be used even in *Windows Subsystem for Linux*.

Installation

► On an Ubuntu machine use the following commands:

① Clone the repository:

```
> git clone https://github.com/merkouris148/parallelepipedonn.git
```

② Create a new anaconda environment:

① Goto the parallelepipedonn directory:

```
> cd ./parallelepipedonn
```

② Create the virtual environment:

```
> conda env create -n pnn-env -f environment.yml
```

③ Test the installation:

①

```
> cd ./bin
```

②

```
> python parallelepipedonn -h
```

Examples

▶ Run a single instance:

- ① Be at the bin directory.
- ② For the Cyclic Bottom-Up Dichotomic algorithm, run:

```
> python parallelepipedonn.py -x ../data/inputs/MNIST/7-1.csv  
-c 7 -nn ../nn_weights/mnist_nn-32.onnx -al c-bu-d
```

- ③ For the Top-Down algorithm, run:

```
> python parallelepipedonn.py -x ../data/inputs/MNIST/7-1.csv  
-c 7 -nn ../nn_weights/mnist_nn-32.onnx -al td
```

▶ Run multiple instances *in parallel*:

- ① Be at the experiments directory.
- ② Run all the MNIST examples on 6 cores, with the Top-Down Algorithm:

```
> python experiments_script.py "../data/inputs/MNIST"  
"../nn_weights/mnist_nn-32.onnx" 6 10000 60 td
```

Everything should be made as simple as possible, but not simpler.
– A. Einstein

Thank you for your time!

Everything should be made as simple as possible, but not simpler.

– A. Einstein

Thank you for your time!
Any questions?